

Checklist

Platform Engineering Four-Pillars

1. Curated Product Approach

- ☐ We treat the platform as a **product**, not just a collection of infrastructure/tools.
- ☐ We have clearly identified the platform's **customers/application developers**.
- ☐ We understand the recurring problems those customers need solved.
- ☐ Platform priorities are based on customer needs rather than purely technical preferences.
- ☐ We have an opinion about **what is in scope and what is not**.
- ☐ We intentionally curate the technologies and workflows exposed to developers.
- ☐ We provide **paved paths** for common use cases.
- ☐ Paved paths cover most recurring developer needs without trying to address every edge case.
- ☐ Developers can leave the paved path when necessary.
- ☐ We identify common capability gaps that justify building new **“railway” platforms**.
- ☐ New platform capabilities are reusable across multiple application teams.
- ☐ We regularly collect developer feedback and adjust the platform accordingly.
- ☐ Success is measured by developer/customer outcomes, not only platform delivery metrics.

2. Software-Based Abstractions

- ☐ The platform contains **software built by platform engineers**, not just documentation and infrastructure configuration.
- ☐ Platform engineers include people with strong software engineering skills.
- ☐ We build abstractions over underlying OSS, cloud, vendor, and internal systems.
- ☐ The platform has a service/API layer where appropriate.
- ☐ APIs simplify interactions between application teams and underlying systems.
- ☐ We do not hide underlying systems unless the abstraction genuinely improves developer productivity.

- ☐ Developers can access underlying primitives when the abstraction would create unnecessary friction.
- ☐ We use **thick clients/libraries** when client-side logic provides meaningful value.
- ☐ Client libraries have a plan for versioning, observability, debugging, and upgrades.
- ☐ We customize OSS where necessary rather than treating upstream products as untouchable.
- ☐ We maintain integrations and extensions required to provide a cohesive platform experience.
- ☐ We maintain metadata describing platform resources and their usage.
- ☐ Metadata can answer questions such as:
 - ☐ Who owns this resource/service?
 - ☐ Who is using it?
 - ☐ What access does it have?
 - ☐ Which team should be charged for its usage?
 - ☐ What dependencies does it have?
 - ☐ What migration impact would changing it create?
- ☐ We have an appropriate metadata mechanism, such as:
 - ☐ Cloud/resource tags
 - ☐ API/schema registry
 - ☐ Service catalog
 - ☐ Internal developer portal
- ☐ Metadata collection is automated wherever possible rather than relying on engineers to manually curate it.

3. Serve a Broad Base of Application Developers

- ☐ The platform is intended for **many application teams**, not only one or two teams.
- ☐ We design for different developer skill levels and use cases.
- ☐ Common workflows are self-service.
- ☐ New users can onboard without routine manual work from the platform team.
- ☐ Users can provision and configure platform capabilities themselves.
- ☐ Common workflows have easy-to-use interfaces.
- ☐ Advanced users can still access lower-level building blocks when necessary.
- ☐ We provide appropriate interfaces such as:
 - ☐ UI
 - ☐ CLI
 - ☐ APIs

- ☐ Libraries/SDKs
- ☐ Configuration-as-code
- ☐ Developers have enough **user observability** to diagnose their own problems.
- ☐ Developers can tell when they are using the platform incorrectly.
- ☐ Developers can distinguish application problems from platform problems.
- ☐ Guardrails prevent common security, compliance, reliability, and cost mistakes.
- ☐ Guardrails provide safe defaults without unnecessarily blocking developers.
- ☐ The platform supports multiple applications efficiently.
- ☐ Shared components are **multitenant** which creates meaningful efficiency.
- ☐ We deliberately choose between shared and per-application components rather than forcing everything into one model.
- ☐ Tenant isolation, reliability, capacity, and security have been considered.
- ☐ The platform becomes cheaper/easier to operate as adoption grows rather than requiring proportional increases in platform-team effort.

4. Operate the Platform as a Foundation

- ☐ Application teams can depend on the platform for important production workloads.
- ☐ The platform is stable enough to function as a **foundation for the business**.
- ☐ The platform team owns the operational experience of the complete offering.
- ☐ We do not push unnecessary operational complexity onto application teams.
- ☐ The platform team takes responsibility for the underlying:
 - ☐ Platform software
 - ☐ Cloud services
 - ☐ OSS systems
 - ☐ Vendor systems
 - ☐ Internal dependencies
- ☐ The team does more than provision infrastructure and hand it over.
- ☐ The team does more than provide thin frameworks or UIs while leaving all operations to developers.
- ☐ Production incidents have clear ownership.
- ☐ Monitoring and alerting exist for critical platform components.
- ☐ SLOs/SLIs or equivalent reliability expectations are defined.
- ☐ Capacity and scalability are actively managed.
- ☐ Backup/recovery requirements are understood where applicable.
- ☐ Operational runbooks exist for important failure modes.
- ☐ Platform engineers participate in operational/on-call responsibilities where appropriate.

- ☐ We routinely test or review operational procedures.
- ☐ We actively investigate anomalies and “unknown unknowns”.
- ☐ User support is treated as part of platform engineering.
- ☐ Application teams know how to obtain platform support.
- ☐ Support questions are used as product feedback.
- ☐ The team maintains strong customer empathy while handling incidents and support requests.

5. Architecture Quality

- ☐ Applications interact with a coherent platform rather than integrating with every underlying system independently.
- ☐ Platform abstractions remove meaningful complexity rather than merely relocating it.
- ☐ The architecture does not force every dependency through a platform API without a clear benefit.
- ☐ The architecture avoids unnecessary coupling between application teams and underlying vendors/OSS.
- ☐ Platform dependencies can be upgraded or migrated without requiring every application team to become an expert.
- ☐ Common functionality is centralized when that improves reliability or efficiency.
- ☐ Application-specific functionality remains outside the platform when centralizing it provides little benefit.

6. Developer Experience

- ☐ Developers have a clear starting point for using the platform.
- ☐ Documentation is easy to discover.
- ☐ Common tasks can be completed without contacting the platform team.
- ☐ Error messages provide users with enough information to resolve problems.
- ☐ Platform workflows fit naturally into developers’ existing working environment.
- ☐ The platform minimizes context switching.
- ☐ Developers do not need deep knowledge of every underlying system.
- ☐ The platform reduces cognitive load.
- ☐ The developer experience is tested with actual platform users.
- ☐ We track adoption, friction, abandonment, or other indicators of developer experience.

7. Internal Developer Portal — Only if Needed

- ☐ We have validated that developers need a centralized portal/catalog.
- ☐ An IDP solves an actual discovery, ownership, self-service, or documentation problem.
- ☐ We are not creating an IDP simply because it is considered fashionable or standard.
- ☐ If we have an IDP, it integrates platform capabilities rather than becoming a disconnected catalog.
- ☐ Platform teams can plug their offerings into a consistent user experience.
- ☐ The portal reduces work rather than adding another interface users must maintain.

8. Cost, Security, and Governance

- ☐ Platform resource ownership is identifiable.
- ☐ Platform usage can be attributed to the appropriate team/product.
- ☐ Access is limited to what services actually require.
- ☐ Default configurations meet organizational security requirements.
- ☐ Compliance requirements are embedded into paved paths where practical.
- ☐ Cost-efficient defaults are provided.
- ☐ Developers can understand the cost implications of major platform choices.
- ☐ Platform-level controls prevent high-impact mistakes without creating unnecessary bureaucracy.

9. AI / ML Platform Considerations

If the platform supports AI workloads:

- ☐ Model-development tooling fits into users' normal working environment.
- ☐ We support the lifecycle of building, training, deploying, and operating models where needed.
- ☐ Compute and storage efficiency are actively optimized.
- ☐ Workload placement helps control infrastructure cost.
- ☐ Model/data access controls are clearly defined.
- ☐ Data provenance is available.
- ☐ We can determine who has access to which data.
- ☐ Model outcomes can be explained to the extent required by the business/regulators.
- ☐ Platform telemetry provides enough data to apply AI/ML to operations.
- ☐ LLM tooling is curated rather than exposing every provider/tool independently.

- ☐ Failure handling, orchestration, debugging, and efficiency are considered part of the AI platform.

Quick Four-Pillar Scorecard

For a fast assessment, score each statement **0 = No**, **1 = Partly**, **2 = Yes**:

Pillar	Question	Score
Product	Are we deliberately curating an offering around developer/customer needs?	☐ 0 ☐ 1 ☐ 2
Development	Are we building software abstractions that genuinely manage complexity?	☐ 0 ☐ 1 ☐ 2
Breadth	Can a broad range of application teams consume it effectively through self-service?	☐ 0 ☐ 1 ☐ 2
Operations	Do we operate and support it as a reliable foundation for the business?	☐ 0 ☐ 1 ☐ 2

Interpretation: **7–8:** strong platform-engineering characteristics; **4–6:** meaningful platform capability, but with important gaps; **0–3:** likely closer to tooling, provisioning, or infrastructure enablement than a mature platform.

The key test running through the chapter is: **Does the platform actually manage complexity for application developers, or does it merely move that complexity somewhere else?**